Symplichain Hackathon - Project Technical Documentation

🔗 Project Overview

This project is an **Interactive Technical Assessment Dashboard** built to present the solutions for the **Symplichain Software Engineering Intern Hackathon**.

Instead of a static PDF, this submission includes a live, interactive React application that demonstrates technical logic through modern UI/UX principles, including real-time navigation and code-artifact management.

🛠️ Technology Stack

The application is built using a modern **Bleeding Edge** frontend stack:

- **Core Framework:** React 19 (Experimental)
- **Build Tool:** Vite 8 (Modern ES Module bundler)
- **Styling:** Tailwind CSS v4 (Using the new `@tailwindcss/vite` engine)
- **Icons:** Lucide-React (Type-safe SVG icon set)
- **State Management:** React `useState` for UI persistence and tab navigation.
- **Typography:** Inter (Google Fonts integration)
- **Animations:** Custom CSS transitions and Tailwind utility classes.

🏗️ Internal Project Structure

The project follows a modular React architecture:

```
/symplichain
├── .github/workflows/      # Automated Deployment Pipelines
│   └── deploy.yml          # GitHub Actions logic
├── src/
│   ├── App.tsx             # Main dashboard logic & route handlers
│   ├── index.css           # Global Tailwind v4 theme configuration
│   └── main.tsx            # Application entry point
├── vite.config.ts          # Vite & Tailwind compilation bridge
├── postcss.config.js       # PostCSS utility configuration
└── package.json            # Dependency & script registry
```

🚀 Key Functional Features

1. Dynamic Multipart Navigation

The dashboard implements an ID-based tab system. Each section of the technical challenge (**Gateway, Mobile, CI/CD, Debugging**) is rendered as a standalone React module, accessible via the sidebar or pagination footer.

2. Live Technical Explainers

- **Interactive Logic:** Part 1 uses visual counters and status bars to explain the "Round-Robin" fairness strategy.

- **Code Artifacts:** Integrated `CommandBlock` components with built-in **Copy-to-Clipboard** handlers for easy command extraction.
- **Workflow Automation:** Ready-to-use GitHub Actions YAML is provided with a dedicated **Download Button** for immediate local use.

3. Responsive "Logistics" UI

The theme is a **High-Contrast Dark Mode** designed for visibility in modern software engineering environments. It includes a mobile-exclusive bottom navigation bar to demonstrate the "mobile optimization" principles discussed in Part 2.

4. Build-Ready Pipeline

The project is configured for **Production-Grade Bundling**. It uses the new Tailwind v4 compiler for lightning-fast CSS processing and produces a minified bundle ready for AWS S3 deployment.

⚙️ How to Run Locally

Prerequisites

- **Node.js:** v20 or later
- **npm:** v10 or later

Installation

1. Navigate to the project root.
2. Install all dependencies:

```
npm install
```

Execution

Start the high-performance Vite dev server:

```
npm run dev
```

The application will be available at http://localhost:5173.

Production Build

To create a minified deployment bundle:

```
npm run build
```

📋 Hackathon Solutions Included

This application provides the detailed technical answers for:

- **Part 1:** Redis-backed request pooling and Token Bucket rate limiting.
- **Part 2:** React Native mobile architecture and SymAI NLP integration.
- **Part 3:** CI/CD pipeline automation with Docker and Terraform migration strategies.

- **Part 4:** Structured 3-phase debugging playbook for Pod upload failure triage.

Submitted by: [Candidate Name] **GitHub Repository:** https://github.com/KalyanRamGoparaboina/Simple-cahin